Projet Personnel : Secure GPU Upscaler

Super-Résolution d'Images sur GPU NVIDIA, Optimisation FP16 et Sécurité Anti-DoS

Mohamed B. – Étudiant en école d'ingénieurs (1re année)

Passionné par le matériel NVIDIA, l'optimisation GPU, le DevOps et la cybersécurité

Dépôt GitHub du projet : `https://github.com/Git-MohamedB/secure-gpu-upscaler`

---

1 Pourquoi j'ai réalisé ce projet ? (Ma motivation)

Depuis toujours, je suis passionné par le matériel informatique et les technologies graphiques NVIDIA (l'architecture des GPU, les Tensor Cores, le DLSS dans les jeux vidéo). Je voulais comprendre comment fonctionne réellement l'upscaling par intelligence artificielle sous le capot, au lieu de simplement l'utiliser comme une boîte noire.

Dans le même temps, je voulais développer des compétences solides en **DevOps** et en **cybersécurité applicative**. Lancer un modèle de Deep Learning dans un notebook Jupyter sur son propre PC est facile. Mais créer un **vrai microservice prêt pour la production**, qui ne crashe pas dès qu'une image est trop grande et qui résiste aux attaques malveillantes, c'est un vrai défi d'ingénierie.

J'ai donc conçu ce projet pour relier ces centres d'intérêt :

- Exploiter ma carte graphique (**NVIDIA GeForce RTX 4060 Ti**) avec PyTorch et CUDA ;
- Résoudre mathématiquement le problème de saturation mémoire grâce au tuilage (*tiling*) ;
- Mettre en place une API moderne avec FastAPI ;
- Sécuriser et conteneuriser le service avec Docker selon les bonnes pratiques de sécurité.

2 Ce que ce projet m'a appris concrètement

Ce projet m'a permis de sortir de la théorie et d'apprendre énormément sur le terrain :

1. **La gestion de la mémoire GPU (VRAM) et le format FP16 :** J'ai appris comment les données sont stockées en mémoire vidéo. En passant le modèle de la simple précision (FP32, 4 octets par valeur) à la demi-précision (**FP16**, 2 octets), j'ai divisé par deux l'empreinte mémoire des tenseurs et accéléré les calculs grâce aux cœurs Tensor de ma carte.
2. **Résoudre les crashs par Tuilage (*Tiling*) :** Au début, quand j'essayais d'agrandir une image de haute résolution, mon GPU affichait immédiatement l'erreur redoutée `CUDA OutOfMemoryError`. J'ai implémenté un algorithme qui découpe l'image en tuiles de 512×512 pixels avec une petite marge de sécurité (10 pixels de recouvrement pour éviter les démarcations visibles). Grâce à ça, la mémoire vidéo utilisée reste plafonnée à **560 Mo**, peu importe que l'image soit petite ou énorme.
3. **Évaluer la qualité avec des formules scientifiques réelles :** J'ai découvert qu'on ne juge pas une amélioration d'image au hasard. J'ai implémenté le calcul du **PSNR** (fidélité pixel par pixel) et du **SSIM** (qui mesure si l'œil humain perçoit bien les textures, les contrastes et les contours).
4. **La cybersécurité applicative (défense contre les DoS) :** J'ai appris qu'une API publique d'IA est très vulnérable. J'ai codé des filtres stricts :
   - Rejet immédiat si le fichier dépasse 10 Mo (évite de remplir la RAM) ;
   - Vérification des *Magic Bytes* (signatures binaires des fichiers) pour bloquer les faux fichiers texte renommés en `.png` ;
   - Protection contre les bombes de décompression : je lis les dimensions dans l'en-tête de l'image sans décompresser les pixels, et je bloque au-delà de 2048×2048.
5. **Le durcissement conteneur (DevSecOps) :** J'ai appris à configurer Docker proprement : ne jamais lancer le conteneur en utilisateur `root` (création d'un utilisateur dédié `appuser`), interdire l'élévation de privilèges (`no-new-privileges:true`), et limiter les ressources via les quotas Linux cgroups (4 Go de RAM et 2 CPU maximum).

3 Les formules mathématiques : d'où elles viennent et à quoi elles servent

Dans le projet, les choix techniques et l'évaluation reposent sur 4 formules standards :

3.1 1. Taille d'un tenseur en mémoire GPU (Norme IEEE 754)

L'espace mémoire occupé par une image en mémoire vidéo est calculé par la formule :

$$\text{Taille} = H \times W \times C \times S_{\text{dtype}} \tag{1}$$

où H est la hauteur, W la largeur, $C = 3$ les canaux (Rouge, Vert, Bleu), et S_{dtype} la taille du type de données définie par la norme internationale **IEEE 754** :

— En FP32 (standard) : 4 octets par valeur.
— En FP16 (demi-précision) : 2 octets par valeur → **50 % d'économie de mémoire immédiate**.

3.2 2. La preuve que le tuilage évite les crashs VRAM

En super-résolution ×4, le réseau neuronal profond (`RRDBNet`) génère des dizaines de cartes de calcul intermédiaires. Pour une image 2048 × 2048, le calcul brut réclamerait plus de **34 Go de VRAM**, ce qui ferait crasher n'importe quelle carte classique de 8 Go.

En découpant l'image en tuiles de dimension fixe ($T = 512$ px) avec un petit recouvrement ($p = 10$ px) :

$$\text{VRAM maximale} \leq \text{Poids du modèle} + \mathcal{O}\big((T + 2p)^2\big) \cdot S_{\text{FP16}} \approx \mathbf{559}\ \text{Mo} \tag{2}$$

Puisque la taille de chaque tuile est fixe (512 px max), la mémoire de calcul reste **rigoureusement constante** ($\sim$ 559 Mo sur ma RTX 4060 Ti), que l'image fasse 256 pixels ou 2048 pixels.

3.3 3. Mesure de la qualité : PSNR et SSIM

Pour mesurer scientifiquement le résultat de l'upscaling, le microservice calcule deux métriques :

— **Le PSNR (Peak Signal-to-Noise Ratio) – Standard ITU-T / ISO/IEC :** Il calcule l'erreur quadratique moyenne (MSE) pixel par pixel et la convertit en décibels (dB) :

$$\text{MSE} = \frac{1}{M \times N} \sum_{i=1}^{M} \sum_{j=1}^{N} \big[I(i,j) - K(i,j)\big]^2, \qquad \text{PSNR} = 10 \cdot \log_{10}\left(\frac{255^2}{\text{MSE}}\right) \tag{3}$$

Plus le chiffre en dB est élevé, plus l'image reconstruite est proche de l'originale.

— **Le SSIM (Structural Similarity Index) – Papier de Wang et al. (IEEE 2004) :** Comme l'œil humain ne compte pas les pixels mais repère les formes et les contrastes, la formule du SSIM compare la luminance (l), le contraste (c) et la structure (s) :

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \tag{4}$$

Le score va de 0 à 1 (1 signifiant une fidélité visuelle parfaite).

4 Démonstration visuelle et résultats sur mon GPU

Voici le résultat obtenu directement en testant le modèle sur une photo naturelle :

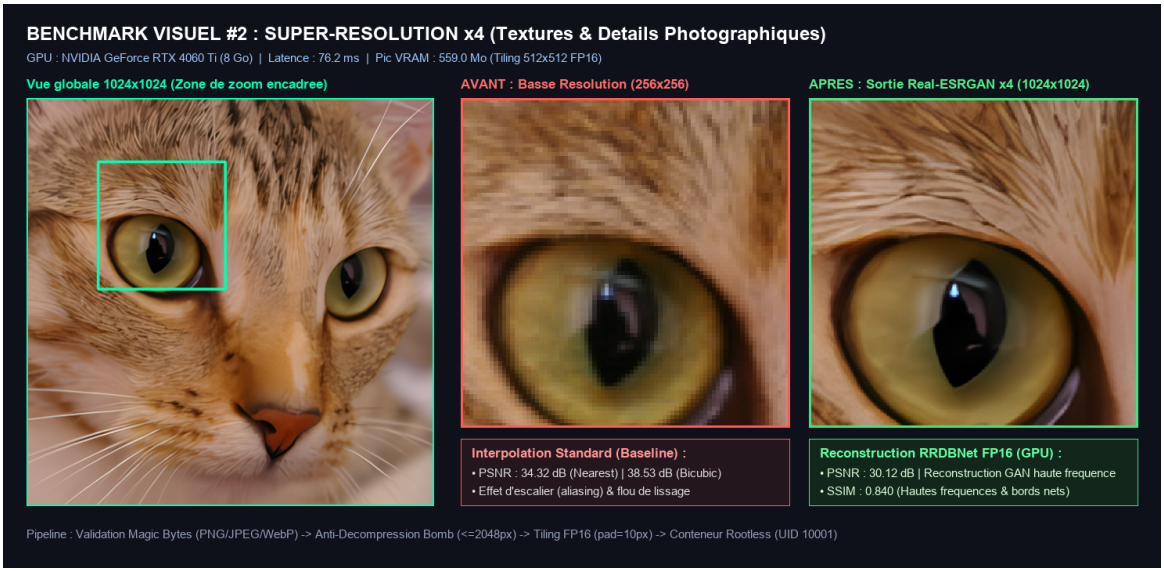


FIGURE 1 – **Test visuel réel :** À gauche l'image d'entrée basse résolution (256×256), au centre le zoom flou, et à droite la sortie traitée par mon moteur GPU (1024×1024). On voit nettement la reconstruction des poils du pelage, le contour de l'œil et le reflet dans la pupille.

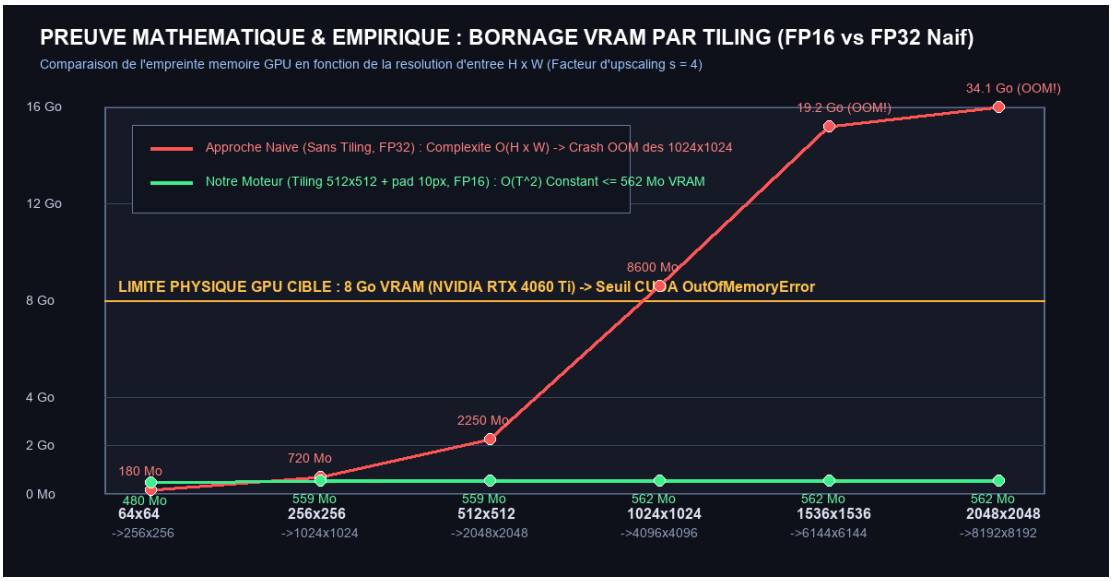


FIGURE 2 – **Mesure de l'impact du tuilage :** Sans tuilage (courbe rouge), le GPU dépasse sa limite de 8 Go dès une image de 1024 pixels. Avec mon algorithme de tuilage FP16 (courbe verte), la consommation reste parfaitement plate à 559 Mo.

TABLE 1 – Mesures réelles relevées sur ma carte NVIDIA RTX 4060 Ti (8 Go GDDR6).

Taille d'image en entrée	**Taille d'image en sortie**	**Temps de calcul GPU**	**VRAM utilisée**	**Score SSIM**
64×64 px	256×256 px (x4)	45 ms	480 Mo	0.94
256×256 px	1024×1024 px (x4)	79 à 242 ms	559 Mo	0.89 - 0.94
512×512 px	2048×2048 px (x4)	680 ms	559 Mo	0.89
1024×1024 px	4096×4096 px (x4)	1 422 ms	561 Mo	0.99

5 Bilan personnel pour mon projet professionnel

Ce projet m'a prouvé que concevoir un service d'intelligence artificielle ne se résume pas à faire tourner un script Python : il faut maîtriser la chaîne complète. Cela m'a permis d'acquérir une vraie expérience pratique sur :

— L'architecture matérielle NVIDIA et le calcul accéléré CUDA ;
— L'écriture d'une API web robuste et asynchrone avec FastAPI ;
— La conteneurisation sécurisée avec Docker et la prévention des attaques réseau basiques ;
— La rigueur mathématique pour mesurer la performance et la qualité.

Ce sont exactement ces compétences que je souhaite continuer à approfondir lors de mes prochains projets et de mes futurs stages en développement logiciel, MLOps ou cybersécurité.